

Core CS - OS



Core CS Interview Prep — OS



Structure

Subject	Top 10	Top 25	Top 50
OS	Most asked, high-priority	Covers core concepts	Deep dive

OS:

→ Top 10:

1. What is the difference between a process and a thread?

→ A process is an independent program in execution with its own memory space, while a thread is a lightweight unit of execution within a process that shares the process's memory. Processes are isolated; threads share heap, code, and data segments but have their own stack and registers. Context switching between threads is faster than between processes.

2. What are the different CPU scheduling algorithms?

→ Common scheduling algorithms include FCFS (First Come First Serve) — simple but causes convoy effect; SJF (Shortest Job First) — optimal average waiting time but requires future knowledge; Round Robin — preemptive, uses a time quantum, good for time-sharing; Priority Scheduling — can cause starvation, solved by aging; and Multilevel Queue Scheduling — separates processes into multiple queues with different policies.

3. What is deadlock? What are the four necessary conditions for it?

→ Deadlock is a state where a set of processes are blocked, each waiting for a resource held by another. The four necessary conditions (Coffman conditions) are: **Mutual Exclusion** (only one process can use a resource at a time), **Hold and Wait** (a process holding a resource waits for more), **No Preemption** (resources can't be forcibly taken), and **Circular Wait** (a circular chain of processes each waiting on the next).

4. What is virtual memory and how does it work?

→ Virtual memory allows a process to use more memory than physically available by using disk space as an extension of RAM. The OS uses paging or segmentation to map virtual addresses to physical addresses via a page table. When a referenced page isn't in RAM, a **page fault** occurs and the OS loads the page from disk (swap space), evicting another page if necessary using algorithms like LRU or FIFO.

5. What is the difference between paging and segmentation?

→ Paging divides memory into fixed-size blocks (pages/frames), eliminating external fragmentation but causing internal fragmentation. Segmentation divides memory into variable-size logical units (segments like code, stack, heap), which is more natural to the programmer but causes external fragmentation. Modern systems often combine both (segmented paging).

6. What is a critical section and how is it protected?

→ A critical section is a code segment that accesses shared resources and must not be executed by more than one process simultaneously. It is protected using synchronization mechanisms. A valid solution must satisfy: **Mutual Exclusion**, **Progress** (if no process is in the critical section, one should be able to enter), and **Bounded Waiting** (a limit on how long a process waits). Tools include mutexes, semaphores, and monitors.

7. What is the difference between a mutex and a semaphore?

→ A mutex (mutual exclusion lock) is a binary locking mechanism where only the thread that acquired the lock can release it — used for ownership-based mutual exclusion. A semaphore is a signaling mechanism with a counter; it can be binary (0/1) or counting (allows N concurrent accesses). Semaphores can be released by a different thread than the one that acquired them, making them suitable for signaling between threads.

8. What are the different page replacement algorithms?

→ When a page fault occurs and memory is full, the OS must evict a page. Key algorithms: **FIFO** — evicts the oldest page, simple but suffers from Belady's anomaly; **LRU (Least Recently Used)** — evicts the page unused for the longest time, good approximation of optimal but costly to implement; **Optimal** — evicts the page not used for the longest time in future,

theoretical best-case baseline; **Clock (Second Chance)** — practical approximation of LRU using a reference bit.

9. What is thrashing?

→ Thrashing occurs when a process spends more time swapping pages in and out than executing, causing CPU utilization to drop sharply. It happens when the system has too many processes competing for limited frames, causing frequent page faults. It can be mitigated using the **Working Set Model** (allocate frames based on the process's active working set) or by reducing the degree of multiprogramming.

10. What is the difference between user mode and kernel mode?

→ The CPU operates in two privilege levels. In **user mode**, processes have restricted access — they cannot directly execute privileged instructions or access hardware. In **kernel mode**, the OS has full access to hardware and memory. A switch from user to kernel mode (a **context/mode switch**) happens via system calls, interrupts, or exceptions. This separation protects the OS from faulty or malicious user programs.

→ **Top 25:**

1. What is an Operating System and what are its main functions?

An OS is system software that manages hardware and software resources. Its main functions include process management, memory management, file system management, I/O device management, and providing a user interface. It acts as an intermediary between users and hardware.

2. What is the difference between a process and a thread?

A process is an independent program in execution with its own memory space, file handles, and resources. A thread is the smallest unit of execution within a process — threads share the same memory space and resources of their parent process. Context switching between threads is faster than between processes.

3. What are the different states of a process?

A process goes through: **New** (being created), **Ready** (waiting for CPU), **Running** (executing), **Waiting/Blocked** (waiting for I/O or an event), and **Terminated** (execution complete). Some models also include a **Suspended** state.

4. What is a deadlock? What are the four necessary conditions for it?

A deadlock is a situation where two or more processes are stuck waiting for each other indefinitely. The four necessary conditions (Coffman conditions) are: **Mutual Exclusion**, **Hold and Wait**, **No Preemption**, and **Circular Wait**. All four must hold simultaneously for a deadlock to occur.

5. **What is the difference between preemptive and non-preemptive scheduling?**

In **preemptive scheduling**, the CPU can be taken away from a running process (e.g., Round Robin, SRTF). In **non-preemptive scheduling**, a process holds the CPU until it voluntarily releases it or terminates (e.g., FCFS, SJF). Preemptive scheduling offers better responsiveness but adds overhead.

6. **Explain virtual memory. How does it work?**

Virtual memory is a memory management technique that allows processes to use more memory than physically available by using disk space as an extension of RAM. It uses **paging** or **segmentation** to map virtual addresses to physical addresses. When a page is not in RAM, a **page fault** occurs and the OS loads it from disk.

7. **What is a page fault? What happens when one occurs?**

A page fault occurs when a process accesses a page that is not currently loaded in physical memory. The OS pauses the process, locates the page on disk, loads it into a free frame in RAM (evicting another page if necessary using a replacement algorithm), updates the page table, and resumes the process.

8. **What are CPU scheduling algorithms? Explain the most common ones.**

- **FCFS**: First-Come-First-Served — simple but can cause the convoy effect.
- **SJF**: Shortest Job First — optimal average waiting time but requires knowing burst times.
- **Round Robin**: Fixed time quantum — good for time-sharing systems.
- **Priority Scheduling**: CPU is given to the highest-priority process — can cause starvation.
- **MLFQ**: Multi-Level Feedback Queue — adjusts priority dynamically.

9. **What is thrashing?**

Thrashing occurs when a process spends more time swapping pages in and out of memory than executing. It happens when the working set of active processes exceeds available physical memory. The OS continuously handles page faults with very little useful work getting done. Solutions include reducing the degree of multiprogramming or using working set models.

10. **What is the difference between internal and external fragmentation?**

Internal fragmentation occurs when allocated memory is slightly larger than requested, wasting space inside the allocated block (common in fixed-size partitioning and paging). **External fragmentation** occurs when free memory exists but is scattered in small, non-contiguous chunks, making it unusable for large allocations (common in dynamic partitioning). Compaction or paging can solve external fragmentation.

11. **What is a semaphore? How does it differ from a mutex?**

A semaphore is a synchronization primitive with an integer value, supporting `wait()` (decrement) and `signal()` (increment) operations. A **mutex** (binary semaphore) is a lock owned by a thread — only the thread that locked it can unlock it. A **counting semaphore** can allow multiple threads to access a resource concurrently. Mutexes enforce ownership; semaphores do not.

12. **Explain the Producer-Consumer problem.**

A classic synchronization problem where a producer generates data and places it in a buffer, and a consumer reads from it. The buffer has a fixed size. The challenge is ensuring the producer doesn't add to a full buffer and the consumer doesn't read from an empty one, without race conditions. Solved using semaphores: `empty`, `full`, and a `mutex`.

13. **What is context switching?**

Context switching is the process of saving the state (context) of a currently running process or thread and loading the state of the next one to run. The context includes the CPU registers, program counter, and process state. It introduces overhead and is triggered by interrupts, system calls, or the scheduler.

14. **What is the difference between paging and segmentation?**

Paging divides memory into fixed-size blocks (pages), eliminating external fragmentation but causing internal fragmentation. **Segmentation** divides

memory into variable-size logical units (segments like code, stack, heap), which matches the programmer's view but can cause external fragmentation. Modern systems often combine both (segmented paging).

15. **What are system calls? Give examples.**

System calls are the interface between a user-space process and the OS kernel. They allow processes to request services like I/O, memory allocation, or process control. Examples: `fork()`, `exec()`, `exit()` (process management), `open()`, `read()`, `write()`, `close()` (file I/O), `malloc()` via `brk()` / `mmap()` (memory).

16. **What is the difference between user mode and kernel mode?**

The CPU operates in two privilege levels. In **user mode**, a process has restricted access to hardware and memory — direct hardware access is not allowed. In **kernel mode**, the OS has full access to all hardware and memory. A transition from user to kernel mode happens via system calls, interrupts, or exceptions. This separation protects the OS from faulty or malicious programs.

17. **What is the Banker's Algorithm?**

The Banker's Algorithm is a deadlock avoidance algorithm. It simulates resource allocation and checks whether the resulting state is **safe** (i.e., there exists a sequence in which all processes can finish). If granting a request leads to an unsafe state, the request is denied. It requires knowing the maximum resource needs of each process in advance.

18. **What is a critical section? How do you protect it?**

A critical section is a code segment that accesses shared resources and must not be executed by more than one process at a time. It must satisfy three properties: **mutual exclusion**, **progress**, and **bounded waiting**. It is protected using synchronization mechanisms like mutexes, semaphores, or monitors.

19. **What is the difference between multiprogramming, multitasking, and multiprocessing?**

- **Multiprogramming:** Multiple programs loaded in memory; CPU is kept busy by switching when one process waits for I/O.
- **Multitasking:** Extension of multiprogramming with rapid CPU switching (time-sharing) to give the illusion of simultaneous execution.

- **Multiprocessing:** Multiple CPUs/cores executing processes truly in parallel.

20. What are the different page replacement algorithms?

- **FIFO:** Replace the oldest page — simple but can suffer from Bélády's anomaly.
- **LRU (Least Recently Used):** Replace the page not used for the longest time — good performance but expensive to implement.
- **Optimal:** Replace the page not used for the longest time in the future — theoretical best, used as a benchmark.
- **Clock (Second Chance):** Approximation of LRU using a reference bit.

21. What is a kernel? What are the types of kernels?

The kernel is the core of the OS, managing hardware directly. Types:

Monolithic kernel (all OS services run in kernel space — fast but large; e.g., Linux), **Microkernel** (only essential services in kernel space — more stable/secure but slower due to IPC; e.g., Mach), **Hybrid kernel** (mix of both; e.g., Windows NT, macOS XNU), and **Exokernel** (minimal abstraction, giving applications direct hardware control).

22. What is inter-process communication (IPC)? What are the methods?

IPC allows processes to exchange data and synchronize. Methods include:

Pipes (unidirectional data stream between related processes), **Named Pipes/FIFOs** (between unrelated processes), **Message Queues** (structured messages via kernel), **Shared Memory** (fastest — processes map the same memory region), **Sockets** (network or local communication), and **Signals** (asynchronous notifications).

23. What is the difference between hard links and soft (symbolic) links?

A **hard link** is a direct reference to the inode of a file — both the original and the link are indistinguishable; deleting one doesn't affect the other. A **soft (symbolic) link** is a pointer that stores the path to the target file — if the original is deleted, the symlink breaks. Hard links cannot span file systems or point to directories; soft links can.

24. What is a race condition? How do you prevent it?

A race condition occurs when the outcome of a computation depends on the unpredictable order of execution of concurrent threads/processes

accessing shared data. Prevention involves: using **mutexes** or **locks** around critical sections, **atomic operations**, **semaphores**, or **monitors**. The goal is to ensure that only one thread accesses the shared resource at a time.

25. What is the difference between a monolithic kernel and a microkernel?

In a **monolithic kernel**, all OS services (file system, drivers, memory management, scheduling) run in a single large block in kernel space. This is fast due to direct function calls but a bug can crash the entire system. In a **microkernel**, only the most essential services (IPC, basic scheduling, memory mapping) run in kernel space; the rest run as user-space services. This improves stability and security but adds IPC overhead.

→ **Top 50:**

1. What is an Operating System?

→ An OS is system software that manages hardware resources and provides services for application programs. It acts as an intermediary between users and hardware, handling process management, memory management, file systems, and I/O operations.

2. What are the main functions of an OS?

→ Process management, memory management, file system management, I/O device management, security and access control, and providing a user interface.

3. What is a process vs a thread?

→ A process is an independent program in execution with its own memory space. A thread is a lightweight unit of execution within a process, sharing the same memory space. Threads are faster to create and context-switch than processes.

4. What is a context switch?

→ The act of saving the state (PCB) of a currently running process and loading the state of the next scheduled process. It enables multitasking but has overhead since no useful work is done during the switch.

5. What is a PCB (Process Control Block)?

→ A data structure maintained by the OS for each process. It stores process state, program counter, CPU registers, memory limits, open file lists, and scheduling info.

6. What are the states of a process?

→ New → Ready → Running → Waiting → Terminated. A process moves between these states based on scheduling decisions and I/O events.

7. What is CPU scheduling?

→ The OS mechanism to decide which ready process gets CPU time. Goals include maximizing CPU utilization, throughput, and minimizing turnaround time, waiting time, and response time.

8. What is the difference between preemptive and non-preemptive scheduling?

→ In preemptive scheduling, the OS can forcibly take the CPU away from a running process (e.g., Round Robin). In non-preemptive, the process runs until it voluntarily gives up the CPU (e.g., FCFS, SJF non-preemptive).

9. Explain FCFS scheduling.

→ First Come First Served — processes are scheduled in arrival order. Simple but suffers from the convoy effect, where short processes wait behind a long one.

10. Explain Round Robin scheduling.

→ Each process gets a fixed time quantum in circular order. Preemptive and fair. Performance depends on quantum size — too small causes excessive context switches; too large behaves like FCFS.

11. What is the Shortest Job First (SJF) algorithm?

→ The process with the smallest next CPU burst is scheduled first. Optimal in terms of average waiting time but requires knowing burst time in advance, which is generally not practical.

12. What is Priority Scheduling and what is starvation?

→ Each process is assigned a priority; the highest-priority process runs first. Low-priority processes may never run — this is called starvation. Solved by aging (gradually increasing priority of waiting processes).

13. What is a deadlock?

→ A situation where a set of processes are each waiting for a resource held by another in the set, causing all to be stuck permanently. Requires four conditions simultaneously: Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

14. What are the four Coffman conditions for deadlock?

→ (1) Mutual Exclusion — resource held exclusively. (2) Hold and Wait — holding one resource while waiting for another. (3) No Preemption — resources can't be forcibly taken. (4) Circular Wait — a circular chain of processes waiting for each other.

15. How can deadlock be handled?

→ Prevention (eliminate one Coffman condition), Avoidance (Banker's algorithm), Detection and Recovery (detect cycles, then kill/rollback processes), or simply Ignorance (Ostrich algorithm, used in most OSes).

16. What is the Banker's Algorithm?

→ A deadlock avoidance algorithm. Before granting a resource, the OS checks if the resulting state is "safe" (i.e., there exists a sequence in which all processes can finish). If not safe, the request is deferred.

17. What is a semaphore?

→ A synchronization primitive. A counting semaphore allows N concurrent accesses; a binary semaphore (mutex) allows only 1. Uses `wait()` (P) and `signal()` (V) operations to control access to shared resources.

18. What is a mutex vs a semaphore?

→ A mutex is for mutual exclusion — only the thread that locked it can unlock it. A semaphore is a signaling mechanism and can be signaled by any thread. Mutexes are for ownership; semaphores are for coordination.

19. What is the critical section problem?

→ A code segment accessing shared resources where only one process should execute at a time. Solutions must satisfy: Mutual Exclusion, Progress, and Bounded Waiting.

20. What is a race condition?

→ When the outcome of a program depends on the relative timing of operations by multiple threads/processes. Prevented by proper synchronization using locks, semaphores, or atomic operations.

21. What is the Producer-Consumer problem?

→ A classic synchronization problem where producers add items to a buffer and consumers remove them. Solved using semaphores: `empty`, `full`, and `mutex` to manage buffer access safely.

22. What is the Readers-Writers problem?

→ Multiple readers can read simultaneously, but a writer needs exclusive access. First readers-writers problem prioritizes readers (writers may starve). Second prioritizes writers (readers may starve).

23. What is the Dining Philosophers problem?

→ Five philosophers alternately think and eat, needing two forks. Illustrates deadlock and starvation in resource sharing. Solutions include resource ordering, arbitrator (waiter), or allowing only $N-1$ philosophers to sit.

24. What is virtual memory?

→ A technique that allows processes to use more memory than physically available by storing parts of the process on disk. The OS uses demand paging to load only required pages into RAM.

25. What is paging?

→ Memory is divided into fixed-size physical frames and logical pages. The OS maintains a page table mapping logical page numbers to physical frame numbers. Eliminates external fragmentation.

26. What is segmentation?

→ Memory is divided into variable-size logical segments (code, stack, heap). Each segment has a base and limit. More intuitive but suffers from external fragmentation.

27. What is a page fault?

→ Occurs when a process accesses a page not currently in RAM. The OS handles it by finding the page on disk, loading it into a free frame, updating the page table, and retrying the instruction.

28. What is thrashing?

→ When a process spends more time handling page faults than executing. Occurs when there are too many processes and not enough frames. Fixed by reducing multiprogramming or using the working set model.

29. What are page replacement algorithms?

→ Used to decide which page to evict when RAM is full. Common ones: FIFO (evict oldest), LRU (evict least recently used), Optimal (evict page not used for longest future — theoretical), Clock (approximation of LRU).

30. What is the difference between internal and external fragmentation?

→ Internal fragmentation: wasted space inside an allocated block (paging). External fragmentation: free memory exists but not contiguous enough to satisfy a request (segmentation). Compaction can reduce external fragmentation.

31. What is a TLB (Translation Lookaside Buffer)?

→ A hardware cache for page table entries. Speeds up address translation by storing recently used page-to-frame mappings. On a TLB hit, no page table lookup needed. TLB miss causes a page table walk.

32. What is the difference between logical and physical address?

→ Logical (virtual) address is generated by the CPU and used by processes. Physical address is the actual location in RAM. The MMU translates logical to physical using the page table.

33. What is a file system?

→ A method to organize, store, retrieve, and manage data on disk. Manages files, directories, and metadata. Examples: ext4 (Linux), NTFS (Windows), FAT32.

34. What is an inode?

→ A data structure in Unix-like file systems storing metadata about a file: permissions, owner, size, timestamps, and pointers to data blocks. The filename is stored in the directory, not the inode.

35. What are hard links vs soft (symbolic) links?

→ A hard link is another directory entry pointing to the same inode. A soft link is a file containing the path to the target. Hard links can't span file systems; soft links can but break if the target is deleted.

36. What is disk scheduling?

→ Optimizes the order of disk I/O requests to minimize seek time. Algorithms: FCFS, SSTF (shortest seek time first), SCAN (elevator), C-SCAN (circular SCAN), LOOK/C-LOOK.

37. What is RAID?

→ Redundant Array of Independent Disks. Combines multiple disks for performance or redundancy. RAID 0 (striping, no redundancy), RAID 1 (mirroring), RAID 5 (striping with distributed parity).

38. What is the difference between monolithic and microkernel architectures?

→ Monolithic kernel runs all OS services (drivers, file system, networking) in kernel space — faster but less stable (e.g., Linux). Microkernel runs minimal code in kernel space and services in user space — more stable but slower due to IPC overhead (e.g., Mach).

39. What is a system call?

→ The interface between user programs and the OS kernel. Allows programs to request OS services (file I/O, process creation, memory allocation) by switching from user mode to kernel mode.

40. What is the difference between user mode and kernel mode?

→ Kernel mode has full hardware access; user mode is restricted. Switching to kernel mode happens via system calls, interrupts, or exceptions. This protects the OS from faulty or malicious user programs.

41. What is an interrupt?

→ A signal to the CPU indicating an event needs attention (I/O completion, timer, hardware error). The CPU pauses current execution, saves state, runs the Interrupt Service Routine (ISR), then resumes.

42. What is DMA (Direct Memory Access)?

→ Allows I/O devices to transfer data directly to/from memory without CPU involvement. The CPU initiates the transfer, the DMA controller handles it, and interrupts the CPU when done. Improves CPU efficiency.

43. What is spooling?

→ Simultaneous Peripheral Operations OnLine. Data is queued to a buffer (spool) before being sent to a slow device like a printer. Allows multiple jobs to be queued without each waiting for the device directly.

44. What is a kernel?

→ The core of the OS that manages hardware resources, runs in privileged mode, and provides essential services like process scheduling, memory management, I/O management, and inter-process communication.

45. What is IPC (Inter-Process Communication)?

→ Mechanisms for processes to communicate and synchronize. Methods include: shared memory (fast, needs synchronization), message passing (safer,

OS-mediated), pipes, sockets, signals, and semaphores.

46. What is the fork() system call?

→ Creates a child process that is a copy of the parent. Returns 0 in child, child PID in parent. Child gets a copy of the parent's address space (copy-on-write). Used to spawn new processes in Unix.

47. What is copy-on-write (COW)?

→ Optimization for fork(). Parent and child initially share the same memory pages. Pages are only copied when either process writes to them. Reduces memory usage and speeds up fork().

48. What is a zombie process?

→ A process that has finished execution but still has an entry in the process table because its parent hasn't called wait() to read its exit status. Consuming no resources except a PID slot.

49. What is an orphan process?

→ A process whose parent has terminated before it has. The OS reassigns it to the init process (PID 1), which periodically calls wait() to clean them up.

50. What is the difference between swapping and paging?

→ Swapping moves entire processes between RAM and disk. Paging moves individual pages. Paging is more granular and efficient; swapping causes higher latency but is simpler to implement.